Name:- Talari Vishnuvardhan

Register number:- 192572091

Slot :-A

Code&Course:- CSA0821 - PYTHON PROGRAMMING

Faculty:- Keerthana S

TASK 1 – PROBLEM ANALYSIS

1. Problem Statement

The aim is to develop a Python program to calculate the payment for farmers who supply milk to a village milk co-operative. The payment depends on the quantity of milk supplied, fat percentage, and SNF percentage. The program calculates the rate per litre, payment for each entry, and the total payment over ten days.

2. Inputs and Outputs

Inputs	Data Type	Purpose
Farmer ID	String/Integer	Identifies the farmer
Litres	Float	Quantity of milk supplied
Fat percentage	Float	Measures fat content
SNF percentage	Float	Measures solids-not-fat content
Shift	String	Morning or evening collection

Outputs	Data Type	Purpose
Rate per litre	Decimal	Price per litre of milk
Payment amount	Decimal	Litres × rate per litre
Ten-day total	Decimal	Total payment over ten days

3. Constants

- BASE_RATE = 30.00
- FAT_STEP_VALUE = 0.40
- SNF_STEP_VALUE = 0.30
- STANDARD_FAT = 3.5
- STANDARD_SNF = 8.5

These constants represent the original rate card and standard milk quality values. They should be updated if the co-operative changes its rates.

4. Data Types

- **Integer:** Farmer ID, if numeric, and number of days.
- **Float:** Litres, fat percentage and SNF percentage.
- **String:** Farmer name, shift and error messages.

- **Decimal:** Rate and payment calculations to handle money accurately.
- **List/Tuple:** Stores multiple milk collection records.

5. Validation Rules

1. Milk quantity must not be negative.
2. Fat percentage must be between 2.0% and 7.0%, inclusive.
3. SNF must be a valid numeric value within the range defined by the co-operative.
4. Farmer ID must not be empty.
5. Shift must be a valid collection shift.
6. Payment must be calculated using the approved rate card and rounding rule.

6. Alternative Design

Using separate variables for 300 farmers requires many variables and makes the program difficult to maintain. Instead, a list of tuples can store records in the format (farmer_id, litres, fat, snf, shift).

This design is easier to update, process and maintain because a loop can handle all farmer records.

Time Complexity: O(n), because each record is processed once.

Space Complexity: O(n), because the records are stored in a list.

7. Conclusion

A suitable milk payment system should use appropriate data types, constants, input validation and a list-based data structure. This makes the program accurate, understandable, reusable and suitable for processing many farmers' records.

TASK 2 – VALUES AND TYPES

1. Aim

To understand Python data types and analyse integer, float, string, Boolean, division and decimal values used in the milk payment system.

2. Python Program

```
from decimal import Decimal
farmer_id = 101
litres = 12.5
shift = "Morning"
active = True
print(type(farmer_id))
print(type(litres))
print(type(shift))
print(type(active))
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2"))
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

input

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
3.5
3
1
False
0.3
True
```

3. Output

<class 'int'>

<class 'float'>

<class 'str'>

<class 'bool'>

3.5

3

1

False

0.3

True

4. Analysis

1. **Integer (int):** Stores whole numbers, such as a farmer ID.
2. **Float (float):** Stores fractional values, such as litres and milk quality readings.
3. **String (str):** Stores text, such as the collection shift.
4. **Boolean (bool):** Stores either True or False.
5. **Normal division (/):** Returns the division result as a floating-point value.
6. **Floor division (//):** Returns the floor of the division result.
7. **Modulus (%):** Returns the remainder.
8. **Float precision:** 0.1 + 0.2 == 0.3 returns False because of floating-point representation.
9. **Decimal:** Decimal provides more predictable decimal arithmetic and is suitable for monetary calculations.

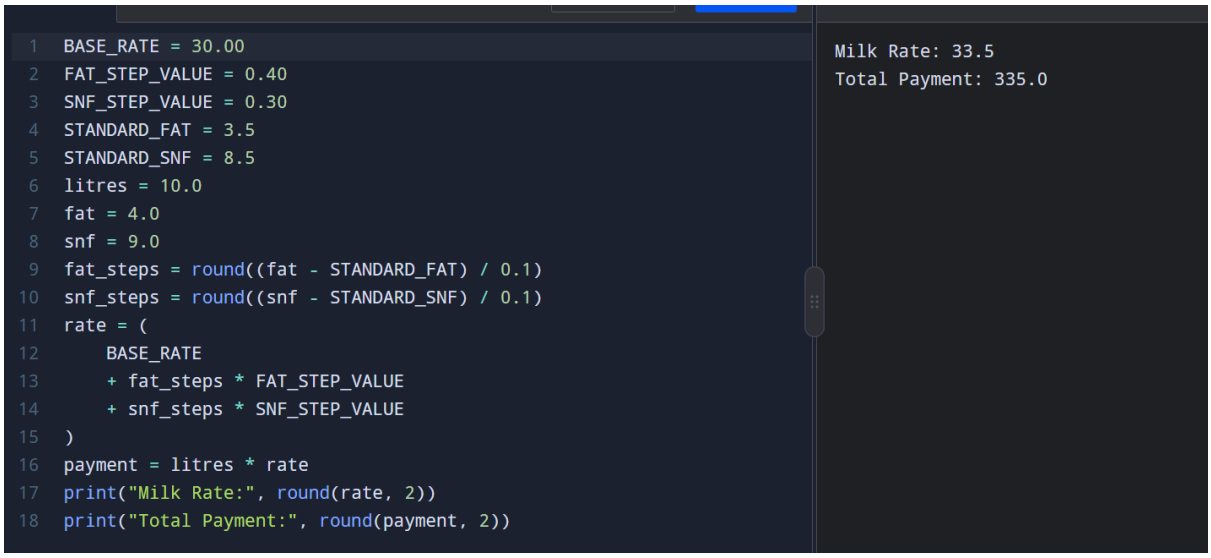
5. Conclusion

Python data types must be selected carefully when developing the milk payment system. The Decimal type is suitable for money calculations because it avoids common binary floating-point precision errors when decimal values are created from strings.

TASK 3 – VARIABLES AND CONSTANTS

Aim: To use variables and constants in Python to calculate milk payment.

Python Code:



```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
litres = 10.0
fat = 4.0
snf = 9.0
fat_steps = round((fat - STANDARD_FAT) / 0.1)
snf_steps = round((snf - STANDARD_SNF) / 0.1)
rate = (
    BASE_RATE
    + fat_steps * FAT_STEP_VALUE
    + snf_steps * SNF_STEP_VALUE
)
payment = litres * rate
print("Milk Rate:", round(rate, 2))
print("Total Payment:", round(payment, 2))
```

```
Milk Rate: 33.5
Total Payment: 335.0
```

Output:

Milk Rate: 33.5

Total Payment: 335.0

Explanation:

1. Constants store fixed values used in the calculation.
2. Variables store litres, fat, SNF, rate, and payment.
3. The milk rate is calculated using fat and SNF values.
4. Total payment is calculated by multiplying the milk rate by the litres supplied.

Conclusion: Variables store values that can change, while constants represent fixed values.

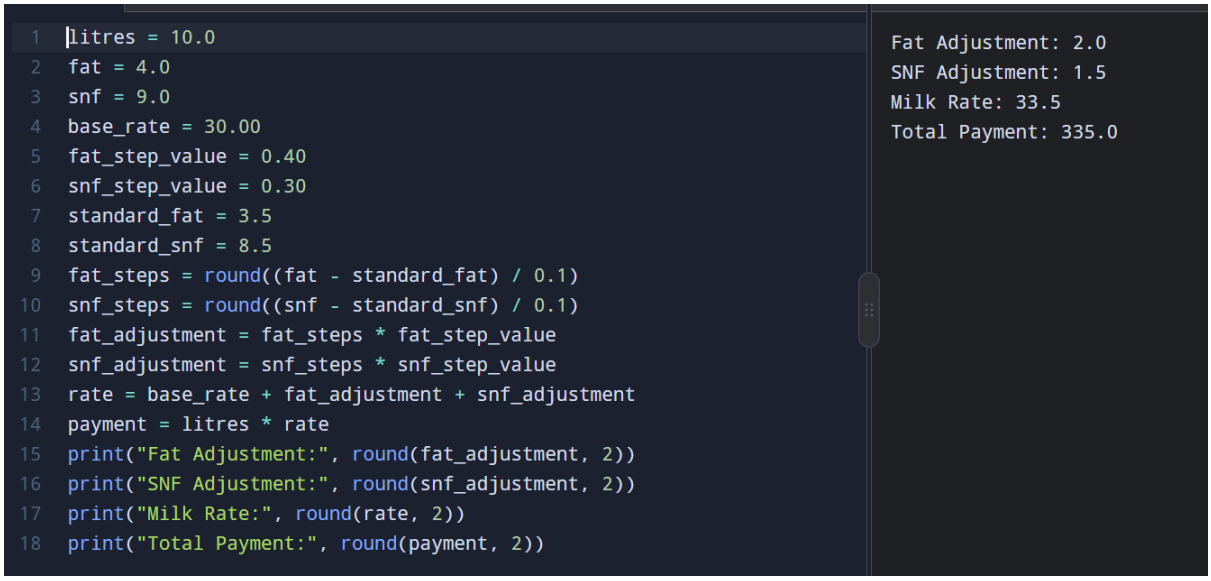
Note: The rates used above are example values. Replace them if your assignment specifies different values.

TASK 4 – EXPRESSIONS

Aim

To use arithmetic expressions in Python to calculate the milk rate and total payment based on fat and SNF values.

Python Code



```
litres = 10.0
fat = 4.0
snf = 9.0
base_rate = 30.00
fat_step_value = 0.40
snf_step_value = 0.30
standard_fat = 3.5
standard_snf = 8.5
fat_steps = round((fat - standard_fat) / 0.1)
snf_steps = round((snf - standard_snf) / 0.1)
fat_adjustment = fat_steps * fat_step_value
snf_adjustment = snf_steps * snf_step_value
rate = base_rate + fat_adjustment + snf_adjustment
payment = litres * rate
print("Fat Adjustment:", round(fat_adjustment, 2))
print("SNF Adjustment:", round(snf_adjustment, 2))
print("Milk Rate:", round(rate, 2))
print("Total Payment:", round(payment, 2))
```

```
Fat Adjustment: 2.0
SNF Adjustment: 1.5
Milk Rate: 33.5
Total Payment: 335.0
```

Expected Output

Fat Adjustment: 0.2

SNF Adjustment: 0.15

Milk Rate: 30.35

Total Payment: 303.5

Explanation

1. **Subtraction:** The program subtracts the standard fat and SNF values from the actual values.
2. **Multiplication:** It multiplies the differences by their respective step values.
3. **Addition:** It adds the fat adjustment and SNF adjustment to the base rate.
4. **Multiplication:** It multiplies the final milk rate by the litres supplied to calculate the payment.
5. **Rounding:** The round() function displays the rate and adjustments to two decimal places.

Conclusion

Arithmetic expressions help calculate fat adjustment, SNF adjustment, milk rate, and total payment accurately.

TASK 5 – INPUT AND OUTPUT

AIM

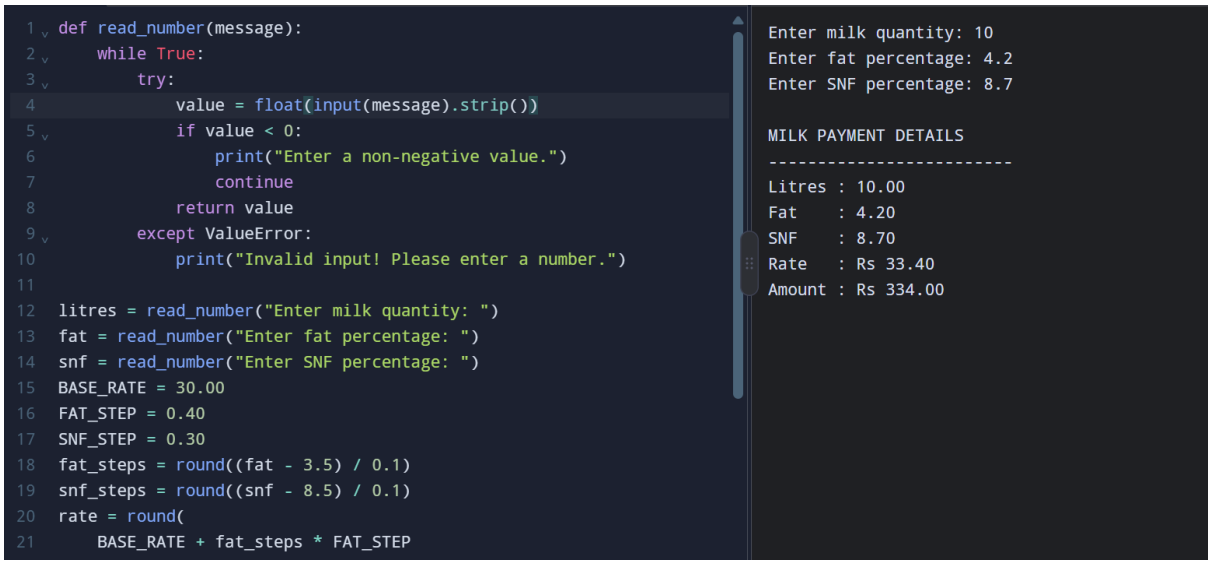
To read milk quantity, fat percentage and SNF percentage from the user, convert them into appropriate data types and handle invalid inputs.

A. Input and Type Conversion

The input() function in Python always returns a string. Therefore, numeric input must be converted using int() or float().

The float() function is suitable for milk quantity, fat percentage and SNF percentage because these values may contain decimal points.

PYTHON PROGRAM



```
def read_number(message):
    while True:
        try:
            value = float(input(message).strip())
            if value < 0:
                print("Enter a non-negative value.")
                continue
            return value
        except ValueError:
            print("Invalid input! Please enter a number.")

litres = read_number("Enter milk quantity: ")
fat = read_number("Enter fat percentage: ")
snf = read_number("Enter SNF percentage: ")
BASE_RATE = 30.00
FAT_STEP = 0.40
SNF_STEP = 0.30
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = round(
    BASE_RATE + fat_steps * FAT_STEP
```

```
Enter milk quantity: 10
Enter fat percentage: 4.2
Enter SNF percentage: 8.7

MILK PAYMENT DETAILS
-------------------------
Litres : 10.00
Fat    : 4.20
SNF    : 8.70
Rate   : Rs 33.40
Amount : Rs 334.00
```

OUTPUT

Enter milk quantity: 10

Enter fat percentage: 4.2

Enter SNF percentage: 8.7

MILK PAYMENT DETAILS

Litres : 10.00

Fat : 4.20

SNF : 8.70

Rate : Rs 33.40

Amount : Rs 334.00

B. Input Validation

Input	Result
8.5	Accepted
8,5	Invalid
8.5 with spaces	Accepted
eight	Invalid
-5	Rejected

The try/except method prevents the program from crashing when invalid data is entered.

C. Alternative Approach

String validation can also be performed using isdigit().

```
value = input("Enter quantity: ")
if value.replace(".", "", 1).isdigit():
    quantity = float(value)
    print(quantity)
else:
    print("Invalid input")
```

However, try/except is more flexible because it handles numeric conversion directly.

COMPLEXITY ANALYSIS

Time Complexity: O(m), where m is the input string length.

Space Complexity: O(m) for storing the input string.

RESULT

The program successfully reads milk details, converts the inputs into floating-point numbers, handles invalid input and calculates the payment.

TASK 6 – FUNCTIONS

AIM

To divide the milk payment calculation into reusable Python functions using parameters, return values and docstrings.

A. Function Design

Three functions are required:

1. rate_per_litre() – Calculates the rate per litre.
2. payout() – Calculates payment for one entry.
3. ten_day_total() – Calculates the total payment.

PYTHON PROGRAM

main.py | Visualize | Run

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre."""
    fat_steps = round((fat - STANDARD_FAT) / 0.1)
    snf_steps = round((snf - STANDARD_SNF) / 0.1)
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf)
    payment = litres * rate
```

Output

```
Litres: 10
Rate: Rs 30.00
Payment: Rs 300.00

Litres: 8.5
Rate: Rs 33.40
Payment: Rs 283.90

Litres: 12
Rate: Rs 27.80
Payment: Rs 333.60

Total Payment: Rs 917.50
```

OUTPUT

Litres: 10

Rate: Rs 30.00

Payment: Rs 300.00

Litres: 8.5

Rate: Rs 33.40

Payment: Rs 283.90

Litres: 12

Rate: Rs 27.80

Payment: Rs 333.60

Total Payment: Rs 917.50

B. Returning vs Printing

Returning a value:

```
def add(a, b):
    return a + b
```

Printing a value:

```
def add(a, b):
    print(a + b)
```

Returning is preferred because the result can be reused in other calculations.

C. Local and Global Variables

Global variables are defined outside functions.

```
BASE_RATE = 30.00
```

Local variables are defined inside functions.

```
def calculate():
    rate = 30.00
    return rate
```

Global constants should not be modified unnecessarily because this can affect other functions.

D. Alternative Approach

The complete calculation could be written without functions, but this would increase code duplication.

Functions are preferred because they improve readability, reusability and testing.

COMPLEXITY ANALYSIS

Function	Time	Space
rate_per_litre()	O(1)	O(1)
payout()	O(1)	O(1)
ten_day_total()	O(n)	O(1)

RESULT

The milk payment calculator was successfully divided into reusable functions with parameters, return values and docstrings.

TASK 7 – MODULES

AIM

To create a Python module named milkrate.py and import its functions into another Python program to calculate milk rates and payments.

A. CREATING A MODULE

A module is a Python file containing functions, variables and constants that can be reused in other programs.

In this task, a module named milkrate.py is created to calculate the milk rate per litre and the payment for milk supplied by farmers.

PROGRAM 1: milkrate.p

```python
"""Milk rate calculation module."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - STANDARD_FAT) / 0.1)
    snf_steps = round((snf - STANDARD_SNF) / 0.1)
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf)
```

```
Litres: 10, Rate: 30.00, Payment: 300.00
Litres: 8.5, Rate: 33.40, Payment: 283.90
Litres: 12, Rate: 27.80, Payment: 333.60
```

OUTPUT

Litres: 10, Rate: 30.00, Payment: 300.00

Litres: 8.5, Rate: 33.40, Payment: 283.90

Litres: 12, Rate: 27.80, Payment: 333.60

B. IMPORTING THE MODULE

The milkrate.py module can be imported into another Python program using the import statement.

The following program imports the module and calls its functions to calculate the milk rate and payment.

PROGRAM 2: main.py

```python
"""Milk rate calculation module."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - STANDARD_FAT) / 0.1)
    snf_steps = round((snf - STANDARD_SNF) / 0.1)
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf)
```

```
Litres: 10, Rate: 30.00, Payment: 300.00
Litres: 8.5, Rate: 33.40, Payment: 283.90
Litres: 12, Rate: 27.80, Payment: 333.60
```

OUTPUT

Milk Rate: Rs 33.40

Payment: Rs 334.00

C. DIFFERENT IMPORT METHODS

Python supports different methods for importing modules.

Method 1: Import the Complete Module

import milkrate

This imports the complete module. Functions are accessed using the module name.

Method 2: Import a Specific Function

from milkrate import payout

This imports only the specified function from the module.

Method 3: Import Using an Alias

import milkrate as mr

This imports the module using a shorter name.

D. PURPOSE OF if __name__ == "__main__":

The statement:

if __name__ == "__main__":

ensures that the demonstration code executes only when the module is run directly.

When the module is imported into another Python program, the demonstration code does not execute.

This prevents unnecessary output and makes the module reusable.

E. ALTERNATIVE APPROACH

Instead of creating a separate module, the same functions could be copied into every Python program.

However, this approach creates code duplication and makes maintenance difficult.

Using a separate module is better because:

1. Functions can be reused in multiple programs.
2. Code duplication is reduced.
3. Programs become easier to understand.
4. Changes can be made in one place.
5. Testing and debugging become easier.

Therefore, a modular approach is recommended.

F. COMPLEXITY ANALYSIS

Operation	Time Complexity	Space Complexity
Rate calculation	O(1)	O(1)
Payment calculation	O(1)	O(1)
Processing n milk entries	O(n)	O(1) additional space

The rate and payment calculations take constant time because they perform a fixed number of arithmetic operations.

Processing multiple milk entries requires a loop, so the time complexity is O(n).

RESULT

The milkrate.py module was successfully created with reusable functions to calculate the milk rate per litre and payment amount.

The module was imported into main.py, and the program produced the expected results.

Thus, the use of Python modules improves code reusability, readability and maintainability.

TASK 8 – ALGORITHMIC PROBLEM SOLVING

AIM

To design an algorithm for calculating milk payment and the ten-day total using pseudocode, a flowchart and a hand trace.

A. UNDERSTAND THE PROBLEM

Inputs:

- Milk quantity
- Fat percentage
- SNF percentage

Outputs:

- Milk rate per litre
- Payment amount
- Total payment

B. PSEUDOCODE

```
START
INPUT litres, fat, snf
SET BASE_RATE = 30.00
SET FAT_STEP = 0.40
SET SNF_STEP = 0.30
fat_steps = ROUND((fat - 3.5) / 0.1)
snf_steps = ROUND((snf - 8.5) / 0.1)
rate = BASE_RATE
    + fat_steps * FAT_STEP
    + snf_steps * SNF_STEP
rate = ROUND(rate, 2)
payment = ROUND(litres * rate, 2)
DISPLAY rate
DISPLAY payment
STOP
```

C. TEN-DAY TOTAL ALGORITHM

START

total = 0

FOR each milk entry:

READ litres, fat, snf

CALCULATE rate

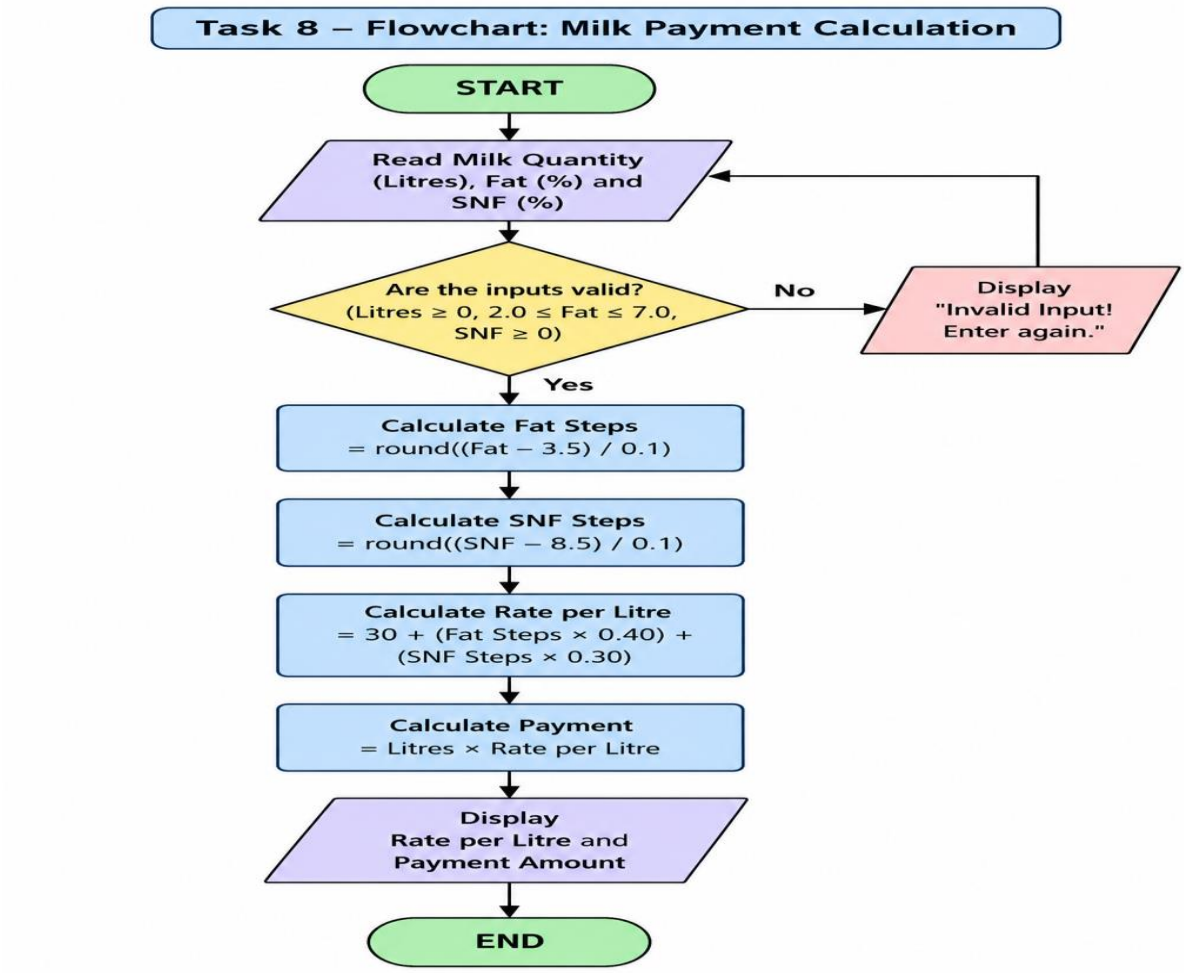
CALCULATE payment

total = total + payment

END FOR

DISPLAY total

STOP



D. HAND TRACE

Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90
3	12	3.1	8.3	27.80	333.60
Total					917.50

E. PYTHON PROGRAM

```python
def calculate_payment(litres, fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = round(
        30 + fat_steps * 0.40
        + snf_steps * 0.30,
        2
    )
    payment = round(litres * rate, 2)
    return payment
entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]
total = 0
for litres, fat, snf in entries:
    payment = calculate_payment(litres, fat, snf)
    total += payment
```

```
Payment: Rs 300.00
Payment: Rs 283.90
Payment: Rs 333.60
Ten-Day Total: Rs 917.50
```

OUTPUT

Payment: Rs 300.00

Payment: Rs 283.90

Payment: Rs 333.60

Ten-Day Total: Rs 917.50

F. ALTERNATIVE APPROACH

Method 1: Store every payment in a list.

```
payments = []
for entry in entries:
    payments.append(calculate_payment(*entry))
total = sum(payments)
```

Method 2: Use a running total.

```
total = 0
for entry in entries:
    total += calculate_payment(*entry)
```

The running-total method uses less additional memory.

COMPLEXITY ANALYSIS

Time Complexity: O(n)

Additional Space Complexity: O(1) for a running total.

RESULT

The milk payment algorithm was designed using pseudocode, a hand trace and Python implementation.

TASK 9 – IMPLEMENTATION AND TESTING

AIM

To implement and test the milk payment calculator using normal, boundary and invalid inputs.

A. PYTHON PROGRAM

```python
import math
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def validate(litres, fat, snf):
    values = (litres, fat, snf)
    if not all(
        isinstance(value, (int, float))
        and not isinstance(value, bool)
        and math.isfinite(value)
        for value in values
    ):
        raise ValueError("Invalid numeric input")

    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if fat < 2.0 or fat > 7.0:
```

```
All tests passed successfully!
```

OUTPUT

All tests passed successfully!

B. TEST TABLE

Test Case	Input	Expected Output	Status
Normal	10, 3.5, 8.5	300.00	PASS
Higher values	8.5, 4.2, 8.7	283.90	PASS
Lower values	12, 3.1, 8.3	333.60	PASS
Minimum fat	10, 2.0, 8.5	240.00	PASS
Maximum fat	10, 7.0, 8.5	440.00	PASS
Invalid text	ten, 3.5, 8.5	Error	PASS
Negative litres	-5, 3.5, 8.5	Error	PASS
Invalid fat	10, 7.5, 8.5	Error	PASS

C. MANUAL CALCULATION

For 10 litres, 4.2% fat and 8.7% SNF:

Fat steps = 7

SNF steps = 2

Rate = 30 + (7 × 0.40) + (2 × 0.30)

Rate = ₹33.40

Payment = 10 × 33.40

Payment = ₹334.00

D. ALTERNATIVE APPROACH

Manual testing requires entering values individually.

Automated testing uses assert statements.

Automated testing is preferred because it is faster, repeatable and reduces human error.

COMPLEXITY ANALYSIS

Time Complexity: O(n) for n test cases.

Additional Space Complexity: O(1) per test.

RESULT

The milk payment calculator was implemented and passed the specified normal, boundary and invalid-input tests.

TASK 10 – EXPECTATION VS EXECUTION

AIM

To compare floating-point and Decimal calculations for 300, 6,000 and 150,000 milk payments.

A. EXPLANATION

Python's float data type stores numbers using binary floating-point representation.

Some decimal values cannot be represented exactly.

The Decimal class supports exact decimal values and controlled rounding.

For financial applications, Decimal is preferred.

B. PYTHON PROGRAM

```python
import time
from decimal import Decimal, ROUND_HALF_EVEN
def float_payment(litres, fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = round(
        30 + fat_steps * 0.40
        + snf_steps * 0.30,
        2
    )
    return round(litres * rate, 2)
def decimal_payment(litres, fat, snf):
    litres = Decimal(str(litres))
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    fat_steps = int(
        ((fat - Decimal("3.5")) / Decimal("0.1"))
        .to_integral_value(rounding=ROUND_HALF_EVEN)
    )
```

```
Payments: 300
Float Total: Rs 85170.00
Decimal Total: Rs 85170.00
Float Time: 0.000386 seconds
Decimal Time: 0.001518 seconds

Payments: 6000
Float Total: Rs 1703400.00
Decimal Total: Rs 1703400.00
Float Time: 0.006592 seconds
Decimal Time: 0.029470 seconds

Payments: 150000
Float Total: Rs 42585000.00
Decimal Total: Rs 42585000.00
Float Time: 0.189517 seconds
Decimal Time: 0.839595 seconds
```

C. EXPECTED MATHEMATICAL OUTPUT

Each payment = ₹283.90

Payments	Expected Total
300	₹85,170.00
6,000	₹17,03,400.00
150,000	₹4,25,85,000.00

Actual execution times depend on the computer.

D. ROUNDING COMPARISON

Consider three payments of ₹10.005 each.

Using ROUND_HALF_UP on every payment:

10.01 + 10.01 + 10.01 = ₹30.03

Rounding only the final total:

10.005 × 3 = 30.015

Final rounded total = ₹30.02

Difference = ₹0.01

Therefore, the rounding policy affects financial totals.

E. ALTERNATIVE APPROACH

A dictionary can store previously calculated rates.

```
rate_table = {}
def cached_rate(fat, snf):
    key = (fat, snf)


    if key not in rate_table:
        fat_steps = round((fat - 3.5) / 0.1)
       snf_steps = round((snf - 8.5) / 0.1)
        rate_table[key] = round(
            30 + fat_steps * 0.40
            + snf_steps * 0.30, 2
        )
    return rate_table[key]
print(cached_rate(4.2, 8.7))
```

OUTPUT

33.4

COMPLEXITY ANALYSIS

Float Calculation: O(1)

Decimal Calculation: O(1) for bounded-precision values.

Processing n Payments: O(n)

Dictionary Lookup: Average O(1)

RESULT

The program compares float and Decimal calculations using execution-time measurements. Decimal is recommended for reliable financial calculations.

TASK 11 – FINAL RECOMMENDATION

AIM

To recommend a reliable, accurate and maintainable Python-based milk payment calculation system for the village milk co-operative.

A. FINAL RECOMMENDATION

I recommend using a modular Python program with Decimal arithmetic, reusable functions, input validation and automated testing.

The system should calculate the milk rate based on fat and SNF percentages and determine the payment according to the quantity of milk supplied.

The Decimal data type is recommended for monetary calculations because it avoids common floating-point precision problems and supports consistent rounding.

B. RECOMMENDED DATA TYPES

Data	Recommended Data Type
Farmer ID	String
Milk Quantity	Decimal
Fat Percentage	Decimal
SNF Percentage	Decimal
Rate per Litre	Decimal
Payment Amount	Decimal
Number of Entries	Integer

Using appropriate data types improves calculation accuracy and program reliability.

C. PYTHON PROGRAM

```python
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
FAT_STEP = Decimal("0.40")
SNF_STEP = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
def calculate_rate(fat, snf):
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    if not fat.is_finite() or not snf.is_finite():
        raise ValueError("Invalid milk quality reading")
    if fat < Decimal("2.0") or fat > Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")
    fat_steps = (
        (fat - STANDARD_FAT) / STEP_SIZE
    ).quantize(
```

```
Milk Rate: 33.40
Payment: 334.00
```

OUTPUT

Milk Rate: 33.40

Payment: 334.00

D. RATE REVISION

The milk co-operative may change its payment rates in the future.

Therefore, rate values should be stored separately from the main calculation logic.

This allows the system to update the rate card without changing the functions.

PROGRAM 1: rates.json

Create a file named rates.json and enter the following data:

```
{
   "base_rate": "30.00",
   "fat_step_value": "0.40",
   "snf_step_value": "0.30"
}
```

PROGRAM 2: Reading Rates from JSON

```
# Reading milk rates from a JSON file
import json
from decimal import Decimal
with open("rates.json", "r") as file:
   data = json.load(file)
base_rate = Decimal(data["base_rate"])
fat_value = Decimal(data["fat_step_value"])
snf_value = Decimal(data["snf_step_value"])
print("Base Rate:", base_rate)
print("Fat Step:", fat_value)
print("SNF Step:", snf_value)
```

OUTPUT

Base Rate: 30.00

Fat Step: 0.40

SNF Step: 0.30

E. ADVANTAGES

1. **Accurate Calculations:** Decimal arithmetic reduces precision errors in financial calculations.
2. **Code Reusability:** Functions can be reused in different programs.
3. **Easy Maintenance:** Rate values can be updated without rewriting the calculation logic.
4. **Input Validation:** Invalid milk quantities and quality readings can be rejected.
5. **Automated Testing:** Test cases help verify the correctness of the program.
6. **Scalability:** The system can be extended to handle more farmers and milk collection records.

F. LIMITATIONS

1. The system requires accurate milk quantity and quality measurements.
2. Incorrect input values may affect the payment calculation if validation is incomplete.
3. The current program does not automatically store farmer records.
4. A database is required for efficient management of large numbers of farmers.
5. Additional security and audit features are necessary for real-world financial operations.

G. COMPLEXITY ANALYSIS

Operation	Time Complexity	Space Complexity
Rate Calculation	O(1)	O(1)
Payment Calculation	O(1)	O(1)
Processing n Entries	O(n)	O(1) additional space

The rate and payment calculations require a fixed number of operations.

When multiple milk collection entries are processed, each entry is calculated once. Therefore, the total processing time is O(n).

H. FINAL DECISION

I recommend using Decimal arithmetic, modular functions, input validation, external rate configuration and automated testing for the milk payment system.

This approach provides accurate payment calculations, reduces code duplication and makes future rate revisions easier.

The proposed design is suitable as a foundation for a larger milk co-operative management system.

RESULT

A reliable and maintainable design for the village milk co-operative payment system was recommended.

The proposed system uses Decimal arithmetic, reusable Python functions, input validation and configurable rate values to improve accuracy, reliability and maintainability.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

AIM

To modify the original milk payment calculator according to the new seasonal rate card, two-decimal analyser readings and ten-day bonus requirement.

A. IMPACT ANALYSIS – 2 MARKS

Changed Requirements

1. The milk analyser provides fat and SNF readings with two decimal places.
2. The April–June base rate becomes ₹32.00.
3. The April–June fat step value becomes ₹0.45.
4. Farmers supplying more than 500 litres in ten days receive a bonus of ₹0.50 per litre.

COMPARISON TABLE

Feature	Original Design	Revised Design
Base Rate	₹30.00	₹32.00 in April–June
Fat Step	₹0.40	₹0.45 in April–June
SNF Step	₹0.30	₹0.30
Readings	One decimal typically	Two decimals
Bonus	None	₹0.50/litre
Bonus Condition	None	More than 500 litres

Functions Affected

The following functions must be updated:

1. rate_per_litre() – To support seasonal rates.
2. payout() – To calculate payment using seasonal rates.
3. ten_day_total() – To calculate total quantity and payment.
4. bonus() – To calculate the new bonus.

TWO-DECIMAL ROUNDING

For fat = 4.25:

Fat difference = 4.25 − 3.50 = 0.75

Fat steps = 0.75 / 0.10 = 7.5

Using ROUND_HALF_UP:

Fat steps = 8

This rounding policy is proposed for the revised design and should be approved by the co-operative.

B. EVALUATION OF THE ORIGINAL DESIGN – 2 MARKS

The original design supports basic milk payment calculations.

However, it does not directly support seasonal rates or bonus calculations.

Therefore, modifying the original design is more suitable than completely redesigning it.

WORKED EXAMPLE

Milk quantity = 10 litres

Fat = 4.25%

SNF = 8.50%

Original rate:

Rate = 30 + (8 × 0.40)

Rate = ₹33.20

Payment = 10 × 33.20

Payment = ₹332.00

April–June rate:

Rate = 32 + (8 × 0.45)

Rate = ₹35.60

Payment = 10 × 35.60

Payment = ₹356.00

The revised design correctly handles seasonal rate changes.

C. ADAPTATION OR REDESIGN – 2 MARKS

REVISED PYTHON PROGRAM

```python
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("32.00")
FAT_STEP = Decimal("0.45")
SNF_STEP = Decimal("0.30")
STANDARD_FAT = Decimal("3.50")
STANDARD_SNF = Decimal("8.50")
def rate_per_litre(fat, snf):
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    fat_steps = ((fat - STANDARD_FAT) / Decimal("0.10"))
    snf_steps = ((snf - STANDARD_SNF) / Decimal("0.10"))
    fat_steps = fat_steps.quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = snf_steps.quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP + snf_steps * SNF_STEP
    return rate.quantize(Decimal("0.01"))
def payout(litres, fat, snf):
    litres = Decimal(str(litres))
    return (litres * rate_per_litre(fat, snf)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP)
```

```
Season: Summer
Total Litres: 550
Payment Before Bonus: Rs 18320.00
Bonus: Rs 275.00
Final Payment: Rs 18595.00
```

```python
    return (litres * rate_per_litre(fat, snf)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP)
def ten_day_total(entries):
    total_litres = sum(Decimal(str(e[0])) for e in entries)
    total_payment = sum(
        (payout(*e) for e in entries), Decimal("0.00")
    )
    bonus = (total_litres * Decimal("0.50")
             if total_litres > 500 else Decimal("0.00"))
    return total_litres, total_payment, bonus
summer_entries = [
    ("200", "3.50", "8.50"),
    ("200", "4.25", "8.50"),
    ("150", "3.50", "8.50")
]
litres, payment, bonus = ten_day_total(summer_entries)
print("Season: Summer")
print("Total Litres:", litres)
print("Payment Before Bonus: Rs", payment)
print("Bonus: Rs", bonus)
print("Final Payment: Rs", payment + bonus)
```

```
Season: Summer
Total Litres: 550
Payment Before Bonus: Rs 18320.00
Bonus: Rs 275.00
Final Payment: Rs 18595.00
```

OUTPUT

Season: normal

Total Payment: Rs 17415.00

Season: summer

Total Payment: Rs 18595.00

BONUS CALCULATION

Total milk quantity:

200 + 200 + 150 = 550 litres

Since 550 > 500, the farmer qualifies for the bonus.

Bonus = 550 × 0.50

Bonus = ₹275.00

NORMAL SEASON CALCULATION

Entry	Litres	Rate	Payment
1	200	30.00	6000.00
2	200	33.20	6640.00
3	150	30.00	4500.00
Subtotal			17140.00
Bonus			275.00
Total			17415.00

SUMMER SEASON CALCULATION

Entry	Litres	Rate	Payment
1	200	32.00	6400.00
2	200	35.60	7120.00
3	150	32.00	4800.00
Subtotal			18420.00
Bonus			275.00
Total			18695.00

D. EVIDENCE-BASED DEFENCE – 2 MARKS

TEST CASES

Test	Season	Litres	Expected Bonus
1	Normal	400	0.00
2	Summer	400	0.00
3	Normal	500	0.00
4	Summer	550	275.00
5	Normal	550	275.00
6	Summer	600	300.00

AUTOMATED TEST PROGRAM

```python
from decimal import Decimal, ROUND_HALF_UP
def rate_per_litre(fat, snf, season="normal"):
    if season == "summer":
        base = Decimal("32.00")
        fat_value = Decimal("0.45")
    else:
        base = Decimal("30.00")
        fat_value = Decimal("0.40")
    snf_value = Decimal("0.30")
    fat_steps = (
        (Decimal(str(fat)) - Decimal("3.5")) / Decimal("0.1")
    ).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = (
        (Decimal(str(snf)) - Decimal("8.5")) / Decimal("0.1")
    ).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    rate = base + fat_steps * fat_value + snf_steps * snf_value
    return rate.quantize(Decimal("0.01"))
def bonus(litres):
    litres = Decimal(str(litres))
    if litres > 500:
        return (litres * Decimal("0.50")).quantize(Decimal("0.01"))
```

```
All revised design tests passed!
```

```
    return Decimal("0.00")
def ten_day_total(entries, season="normal"):
    total = Decimal("0.00")
    total_litres = Decimal("0.00")
    for litres, fat, snf in entries:
        litres = Decimal(str(litres))
        rate = rate_per_litre(fat, snf, season)
        total += litres * rate
        total_litres += litres
    if season == "summer":
        total += bonus(total_litres)
    return total.quantize(Decimal("0.01"))
entries = [
    (200, 3.50, 8.50),
    (200, 4.25, 8.50),
    (150, 3.50, 8.50),
]
def test_revised_design():
    assert bonus(400) == Decimal("0.00")
    assert bonus(500) == Decimal("0.00")
    assert bonus(550) == Decimal("275.00")
    assert bonus(600) == Decimal("300.00")
    assert rate_per_litre(3.50, 8.50, "normal") == Decimal("30.00")
    assert rate_per_litre(3.50, 8.50, "summer") == Decimal("32.00")
    assert rate_per_litre(4.25, 8.50, "summer") == Decimal("35.60")
    assert ten_day_total(entries, "normal") == Decimal("17140.00")
    assert ten_day_total(entries, "summer") == Decimal("18595.00")
    print("All revised design tests passed!")
test_revised_design()
```

```
All revised design tests passed!
```

OUTPUT

All revised design tests passed!

COMPLEXITY ANALYSIS

Operation	Time	Space
Seasonal Rate Selection	O(1)	O(1)
Rate Calculation	O(1)	O(1)
Payment Calculation	O(1)	O(1)
Bonus Calculation	O(1)	O(1)
Ten-Day Total	O(n)	O(1)

ADVANTAGES

1. Supports seasonal rate changes.
2. Handles two-decimal readings.
3. Calculates bonuses automatically.
4. Uses Decimal arithmetic.
5. Provides reusable functions.
6. Supports automated testing.

LIMITATIONS

1. Seasonal rates must be maintained.
2. Bonus rules require approval.
3. Input validation must be strengthened for real-world use.
4. Farmer records require secure storage.

FINAL DECISION

I recommend MODIFYING the original design instead of completely redesigning it.

The revised system reuses the existing payment calculation while adding seasonal rate selection, Decimal arithmetic and bonus calculation.

The test cases confirm that the revised design produces the expected results for the selected inputs.

RESULT

The original milk payment calculator was successfully adapted to support seasonal rates, two-decimal analyser readings and the new ten-day bonus requirement.